

AI and Advanced Machine Learning

Module IX: Foundations of Generative Modeling

Yan Kyaw Tun

Tenure Track Assistant Professor

Edge Computing and Networking Group



AALBORG
UNIVERSITY

Types of Deep Learning Models

Which face is real?



A



B



C

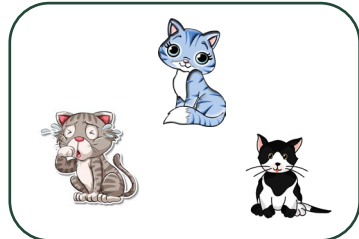
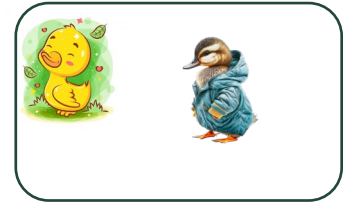
Types of Deep Learning Models



Predictive Model



Not a Duck



Types of Deep Learning Models

- **Discriminative:** Use to classify and Predict
- **Generative:** Generate new data that is similar to the data it was trained

**Discriminative
Technique**



Classify



**Discriminative model
(classify as a dog or a cat)**



**Generative
Technique**



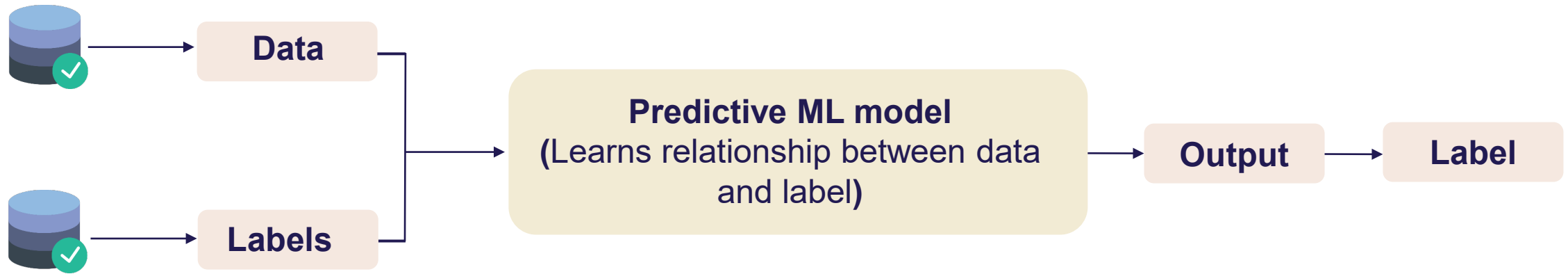
Generate



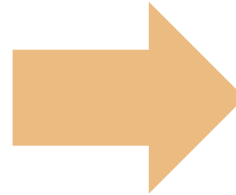
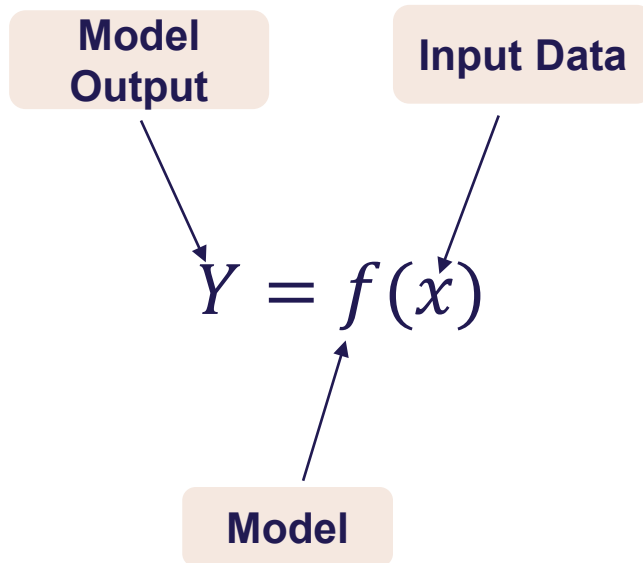
**Generative model
(Generate dog image)**



Types of Deep Learning Models



Types of Deep Learning Models



Not Gen AI if Y is:

- Number
- Discrete
- Class
- Probability

Gen AI if Y is:

- Natural Language
- Image
- Audio

Generative AI (GenAI)

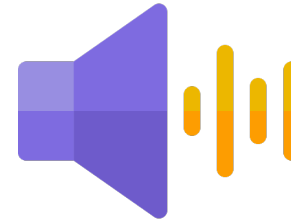
➤ A type of AI that can produce various types of



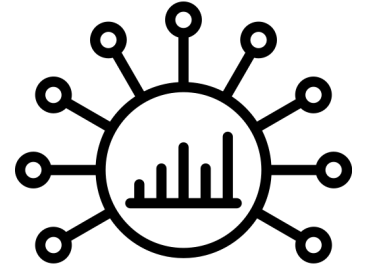
Content



Text and Imagery



Audio



Synthetic Data

Generative AI (GenAI)

- GenAI is a type of artificial intelligence that creates new content based on what it has learned from the existing content
- The process of learning from the existing content is called training and results in the creation of a statistical model
- When given a prompt, GenAI uses this statistical model to predict what an expected response might be-and this generates new content

Generative language models

Generative language models learn about patterns in language through training data.

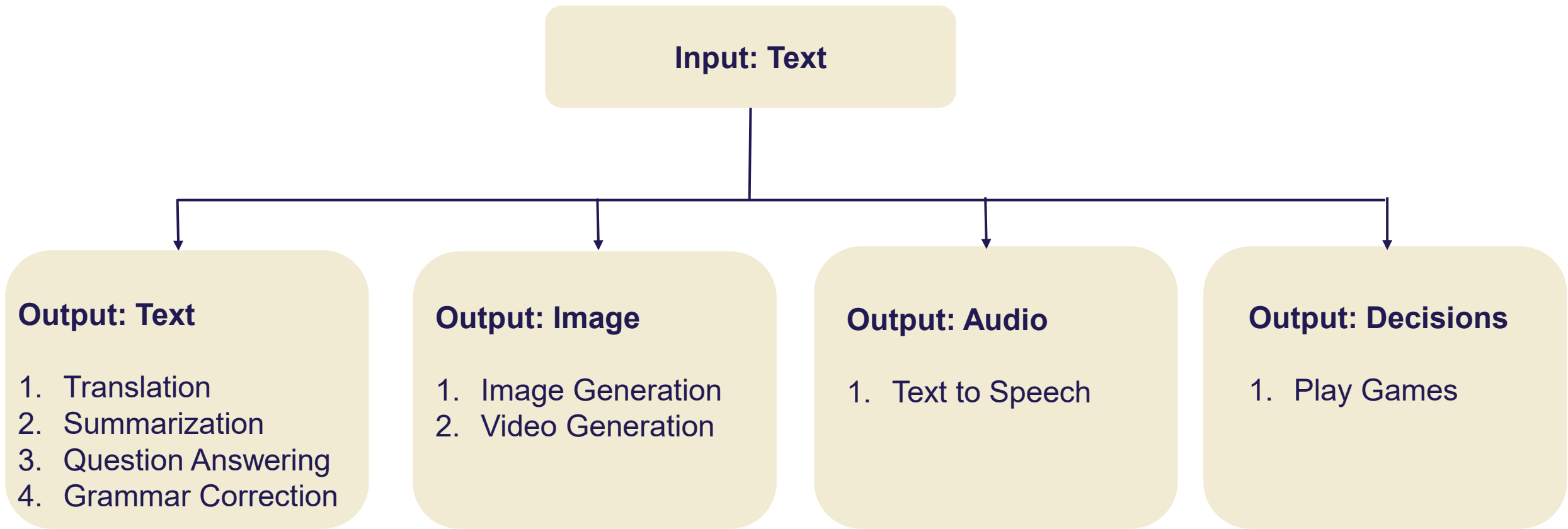
Then, given some text, they predict what comes next.

Generative image models

Generative image models produce new images using techniques like diffusion.

Then, given a prompt or related imagery, they transform random noise into images or generate images from photos.

Generative AI (GenAI)



Gemini

I'm making a sandwich with peanut butter and



Sure I can help you with that. Here are some of the most popular spreads to put on a peanut butter sandwich:

- Jelly or jam: strawberry, grape, raspberry, apricot, peach, blackberry, blueberry, etc.

Gemini

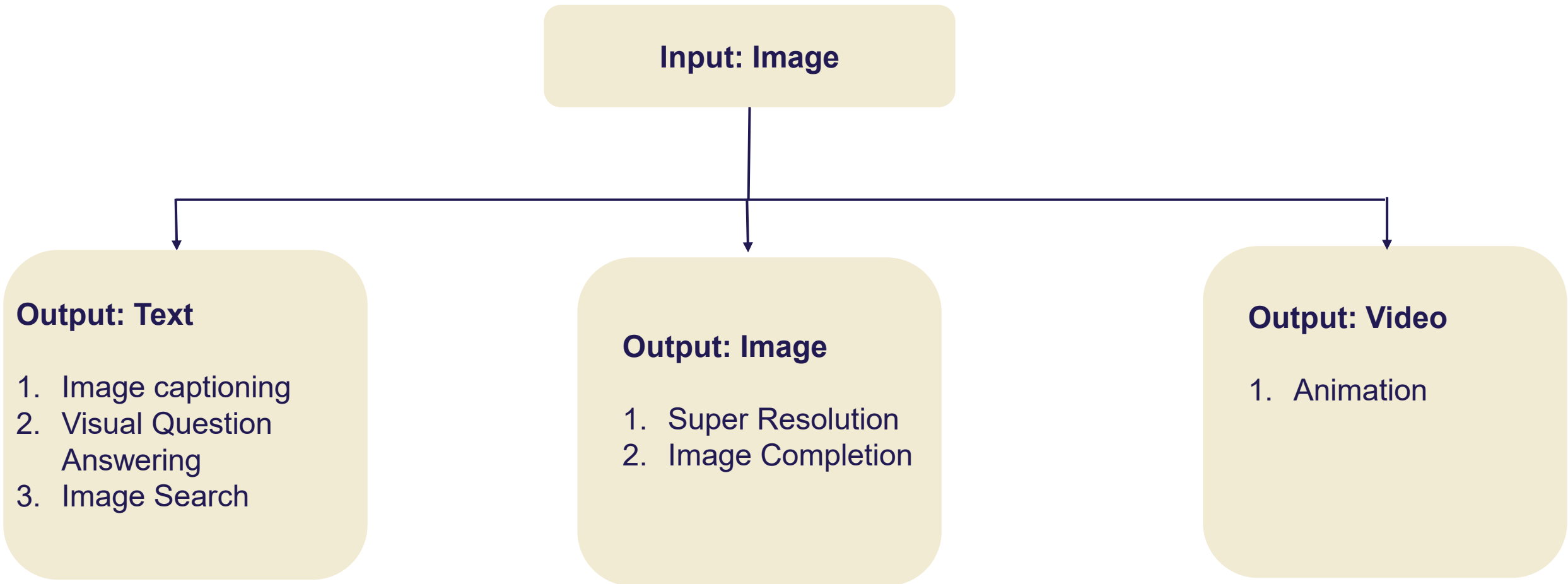
The meaning of life is



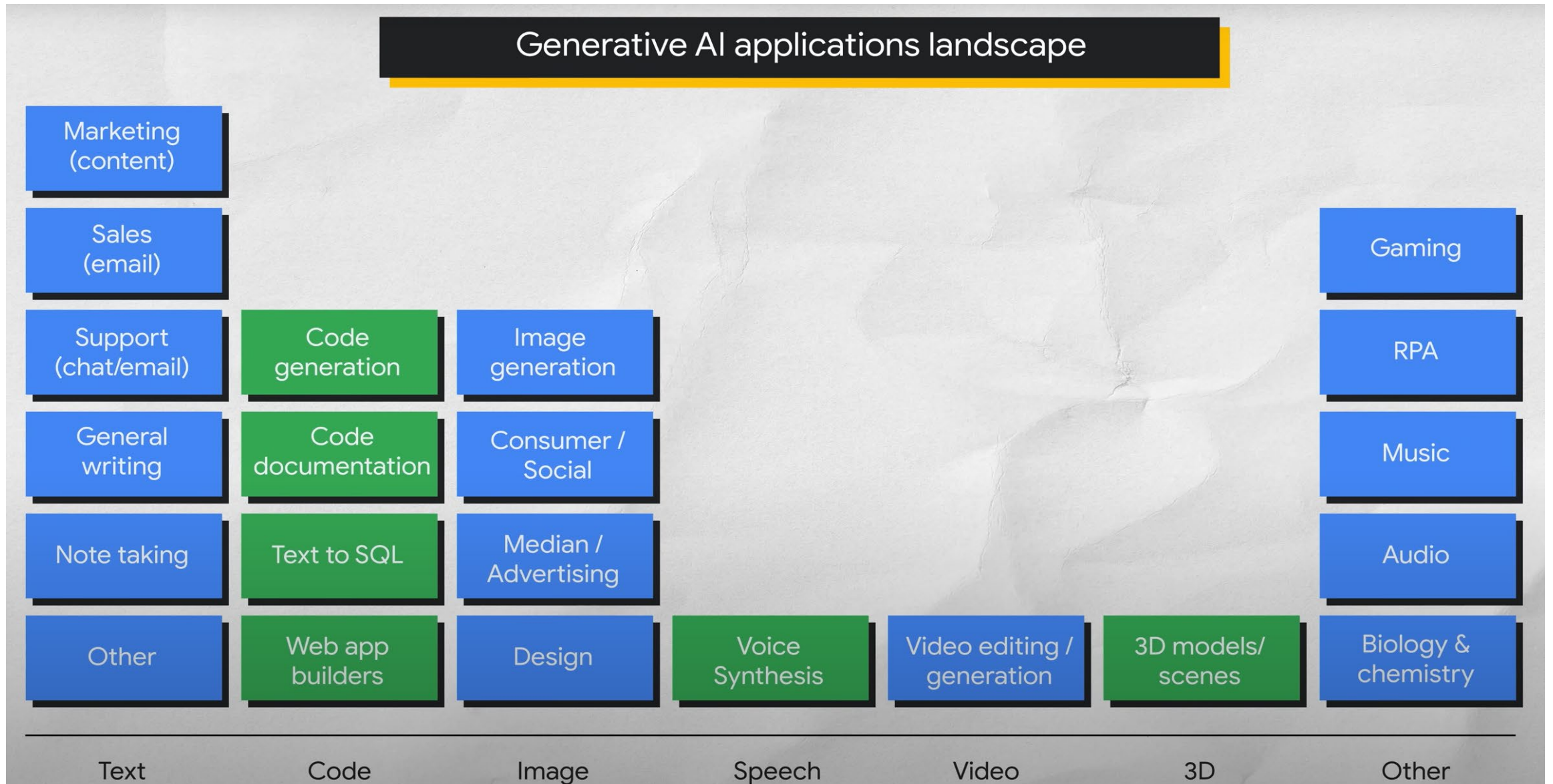
The meaning of life is a question that has been pondered by philosophers and theologians for centuries. There is no one answer that will satisfy everyone, but some possible answers include:

- To find happiness and fulfillment.
- To make a difference in the world.
- To learn and grow as a person.
- To connect with others and build relationships.

Generative AI (GenAI)



Generative AI (GenAI)



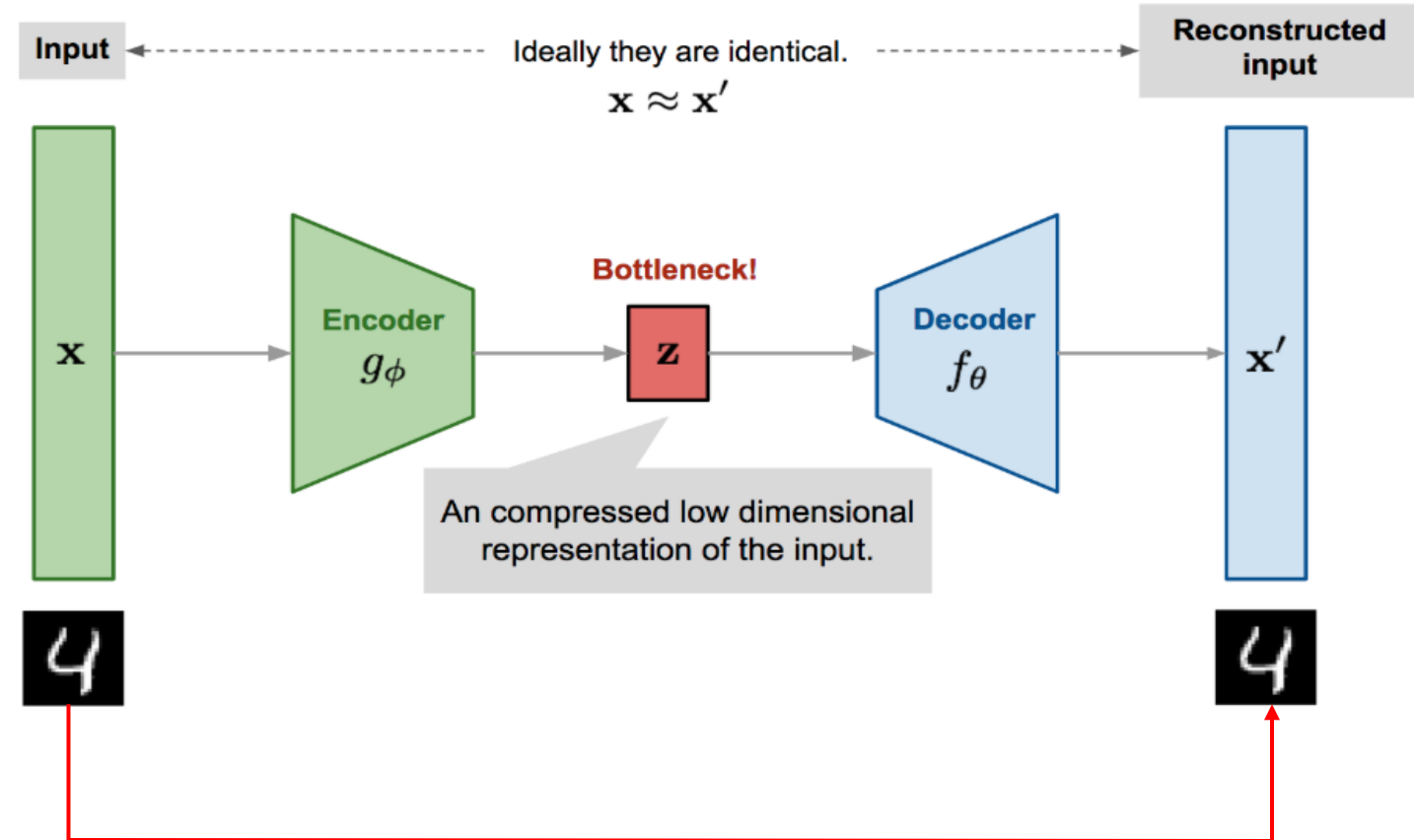
Generative AI (GenAI)

Challenges

- **Data and Training:** Models require massive datasets, leading to high training costs and potential exposure of sensitive, personal information
- **Performance and Reliability:** Hallucinations, inconsistency, limited knowledge
- **Ethical and Legal Risks:** copyright infringement, misuse
- **Technical Limitations:** context management, “black box” problem
- **Sustainability:** High energy consumption for training and running models

Sample Generative Model: Autoencoder

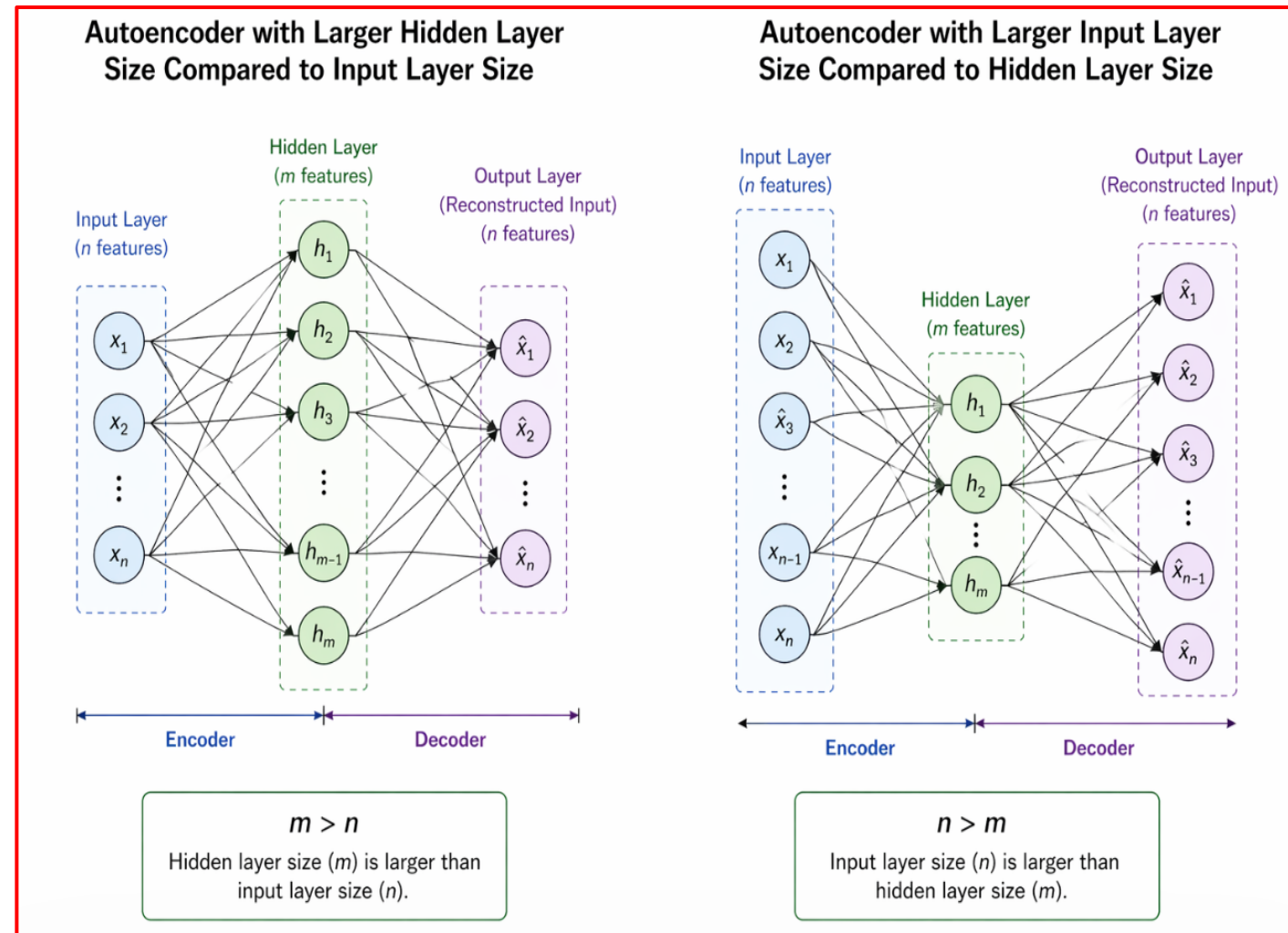
- An autoencoder is a neural network that learns to compress data into a lower-dimensional representation and then reconstruct it
- **Encoder** g_ϕ maps input to an **embedded representation**
- **Decoder** f_θ maps back to the input space



$$\text{Reconstruction Loss: } L(x, x') = \|x - x'\|^2$$

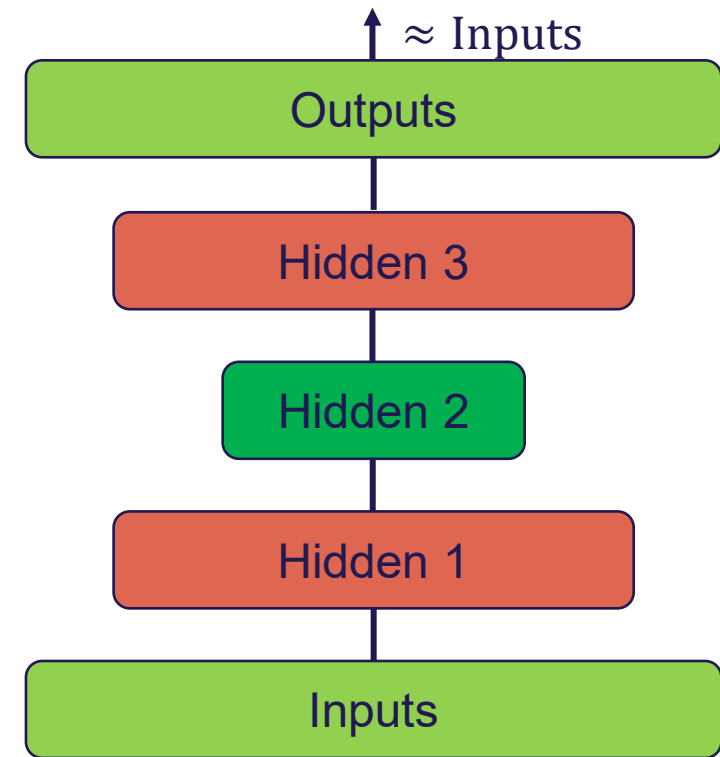
Sample Generative Model: Autoencoder

- Undercomplete autoencoders have hidden layer size smaller than input layer size
 - ✓ Dimension of embedded space is lower than that of the input space
 - ✓ Cannot simply memorize training input
- Overcomplete autoencoders have larger hidden layer sizes
 - ✓ Regularize to avoid overfitting



Stacked Autoencoder

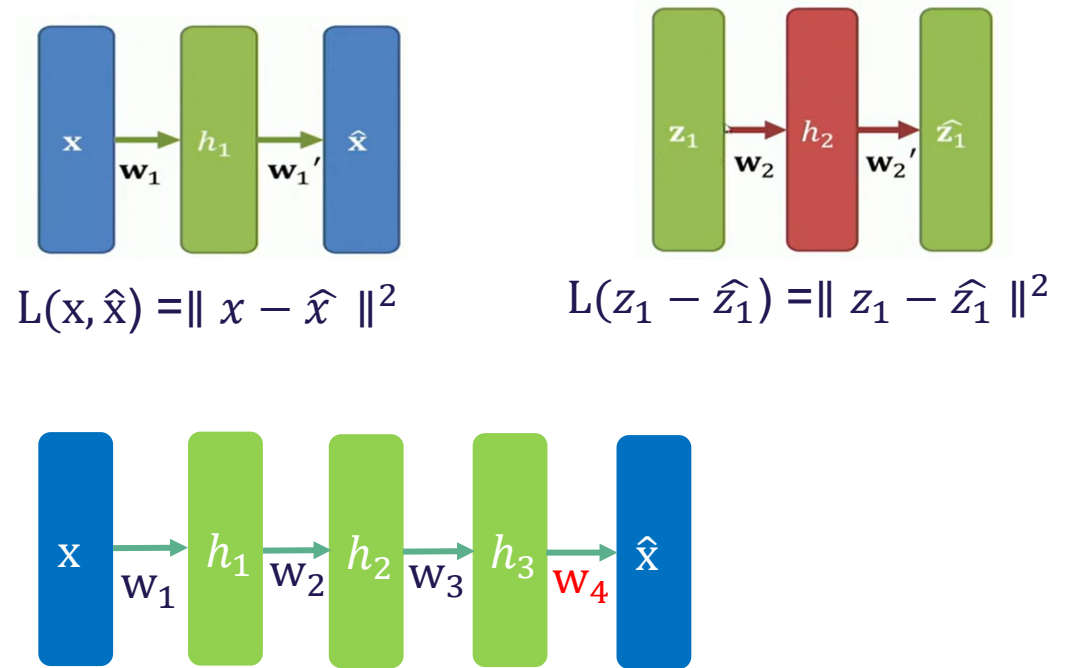
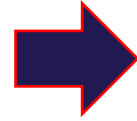
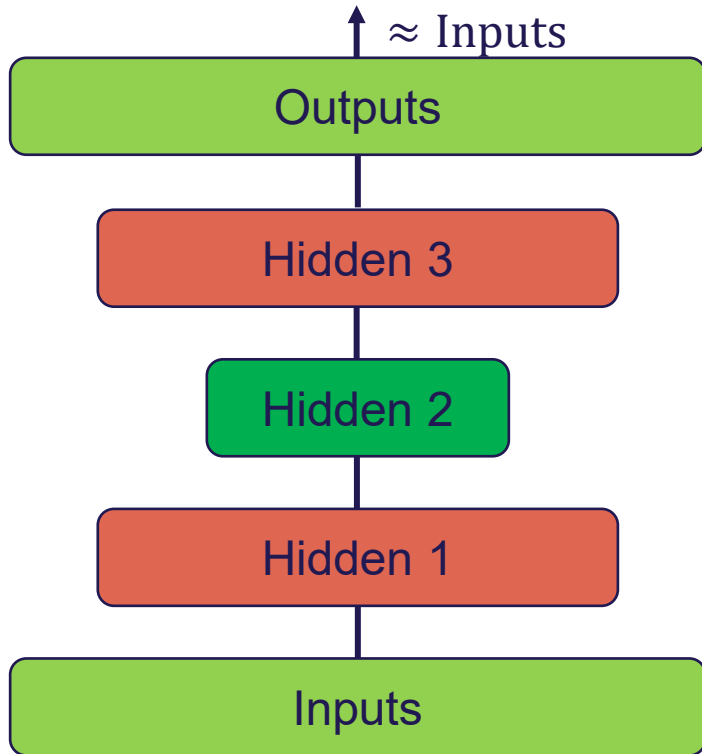
- Stacked autoencoder has multiple hidden layers
- Can share the parameters to reduce their number by exploiting symmetry: $\mathbf{W}_4 = \mathbf{W}_1^T$ and $\mathbf{W}_3 = \mathbf{W}_2^T$



```
weights1 = tf.Variable(weights1_init, dtype=tf.float32, name="weights1")
weights2 = tf.Variable(weights2_init, dtype=tf.float32, name="weights2")
weights3 = tf.transpose(weights2, name="weights3")          # shared weights
weights4 = tf.transpose(weights1, name="weights4")          # shared weights
```

Stacked Autoencoder

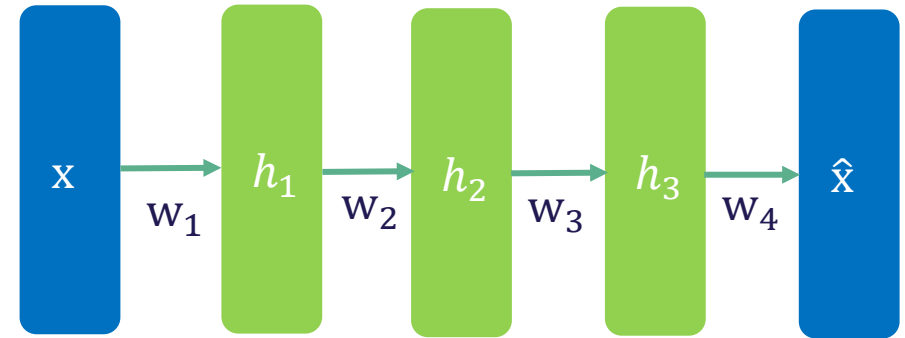
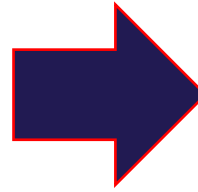
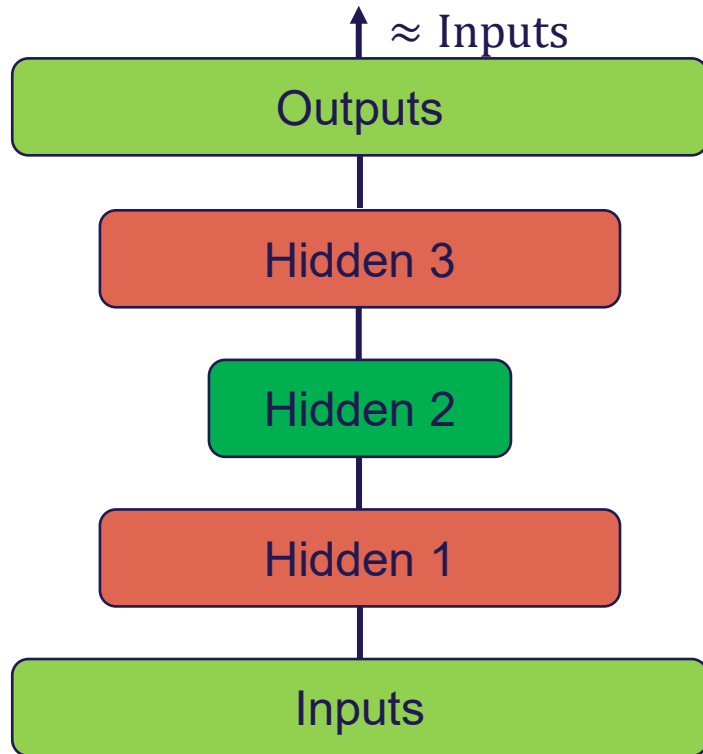
➤ Incremental Training (Layer-wise Training)



- Avoid bad local minima
- Acts as a form of regularization
- Helps with “garbage in, garbage out” issue

Stacked Autoencoder

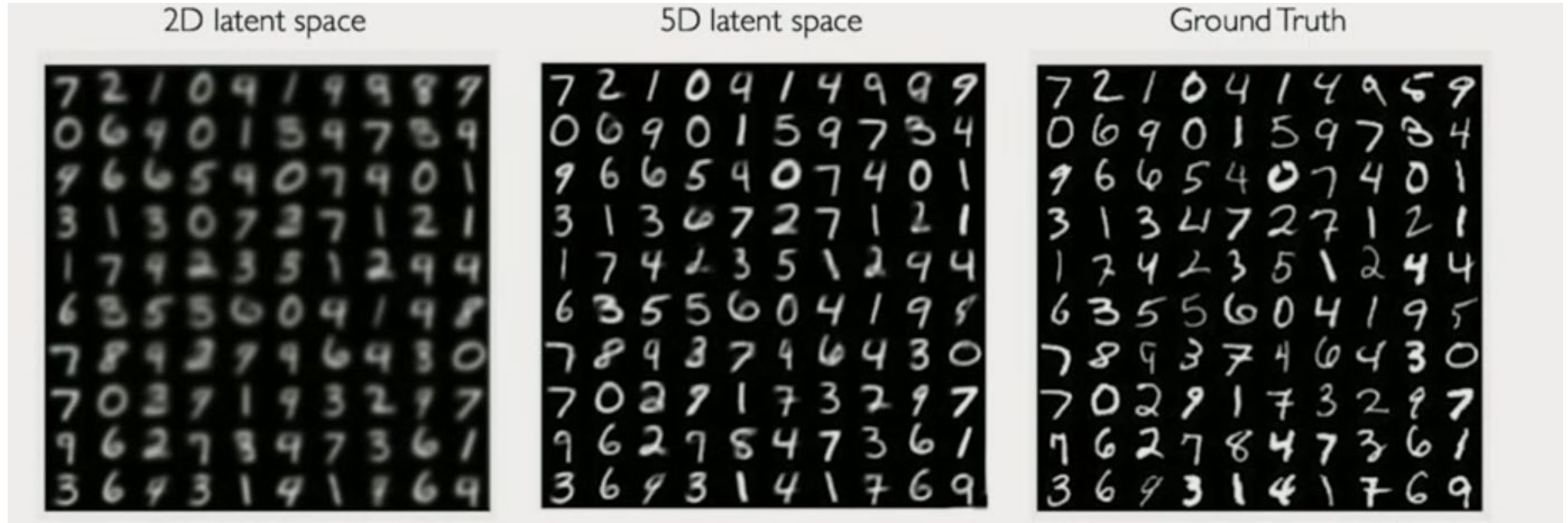
➤ End-to-End Training



$$L(x, \hat{x}) = \|x - \hat{x}\|^2$$

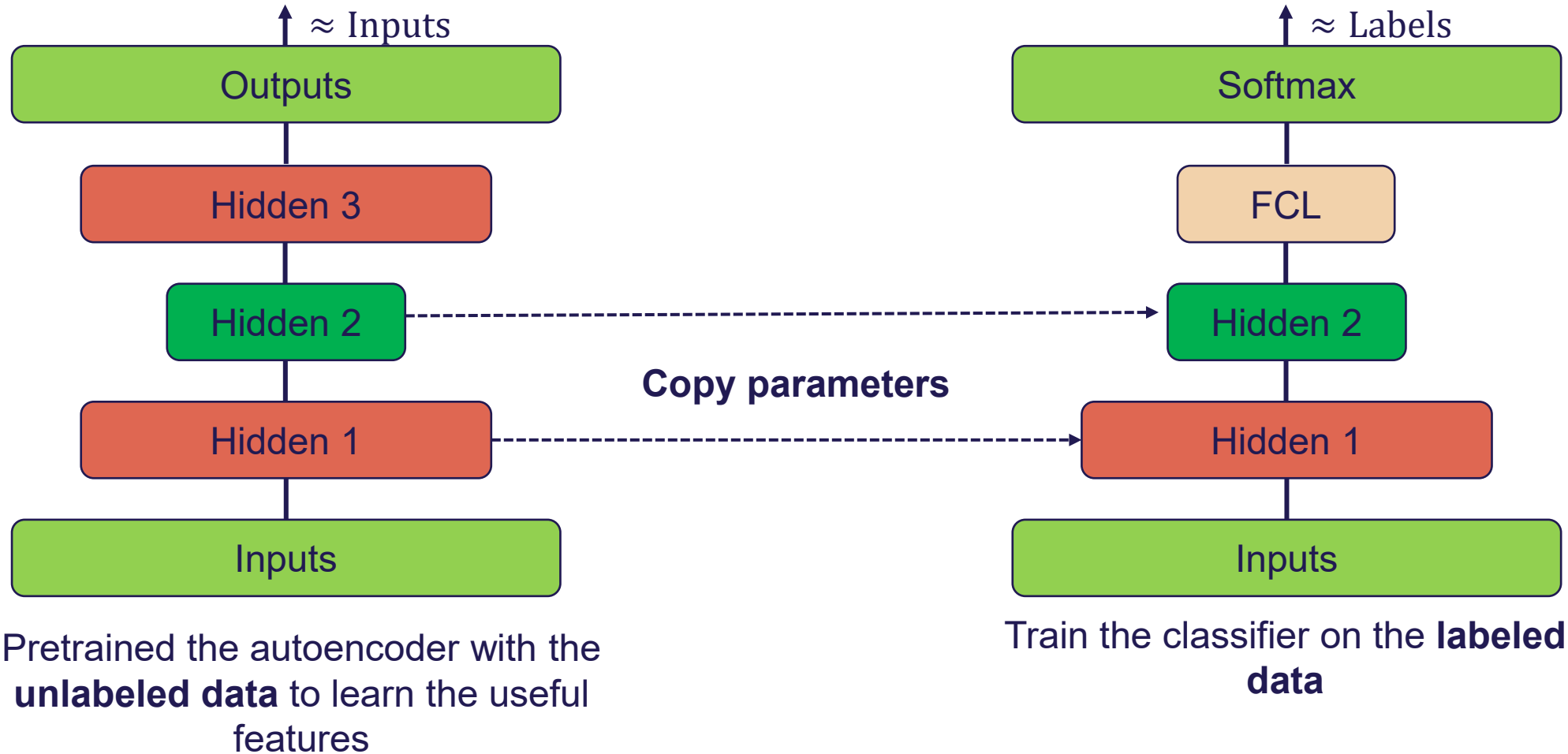
Stacked Autoencoder

- Smaller latent space will force a larger training bottleneck



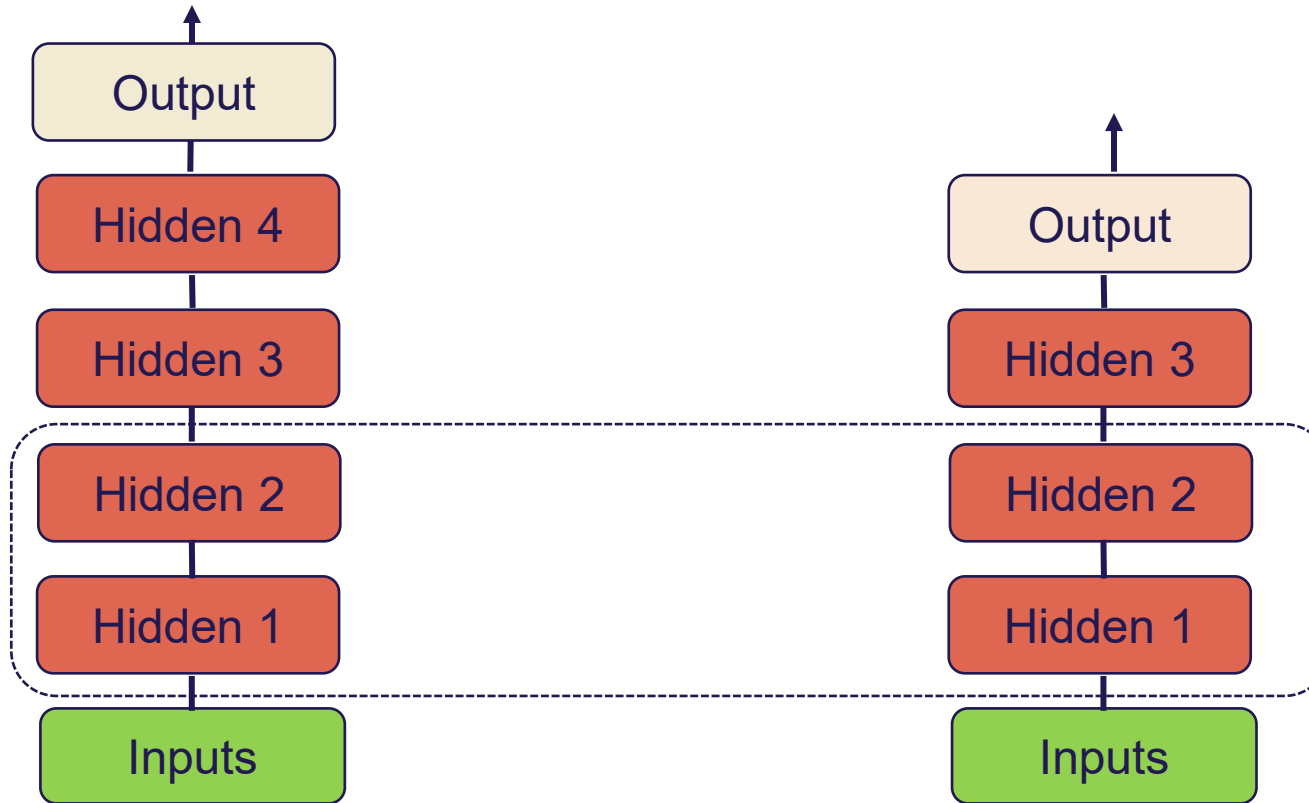
Stacked Autoencoder

Example Application: Semi-Supervised Learning



Stacked Autoencoder

Example Application: Can also transfer from different task

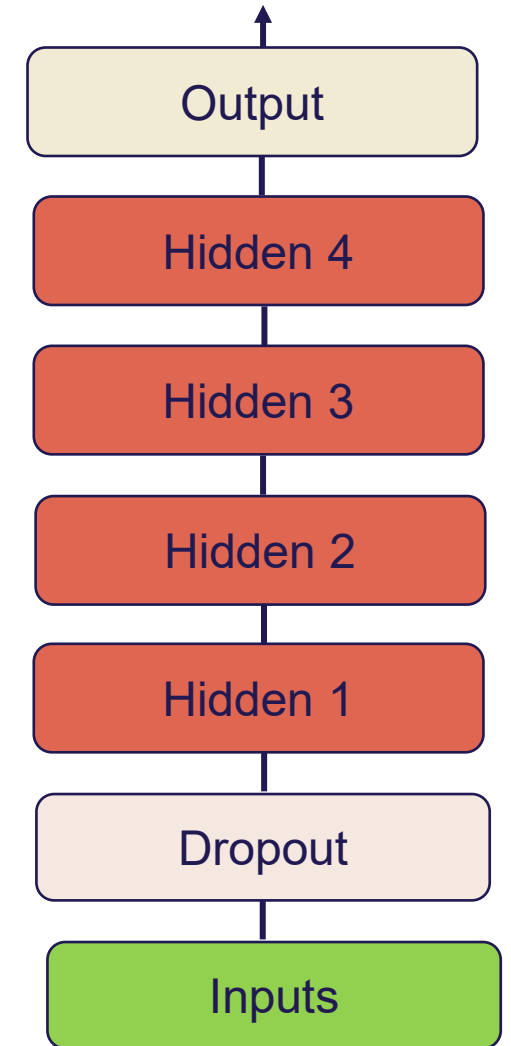
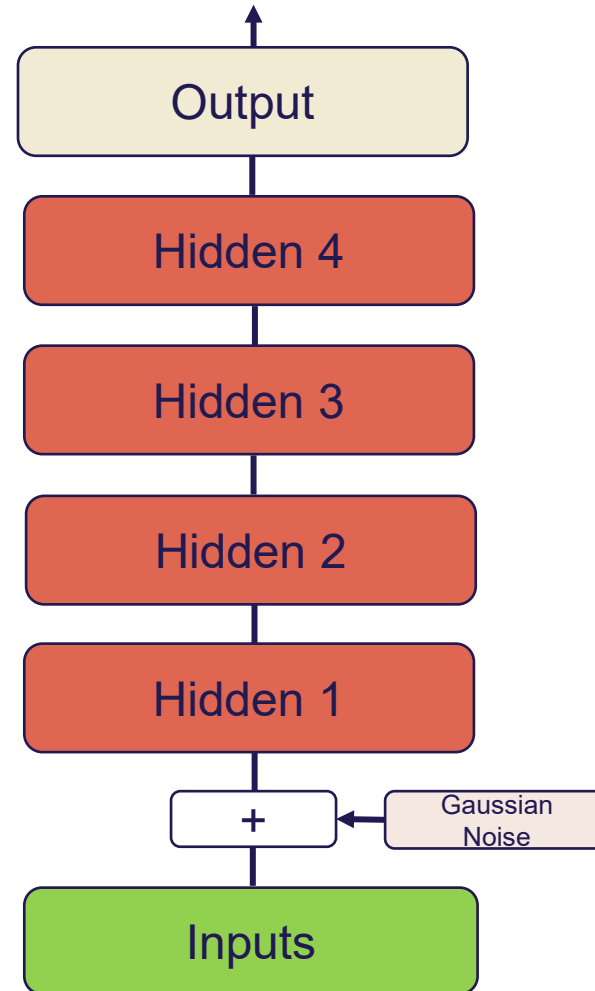


Example: GoogleNet architecture (can download from **model zoo**)

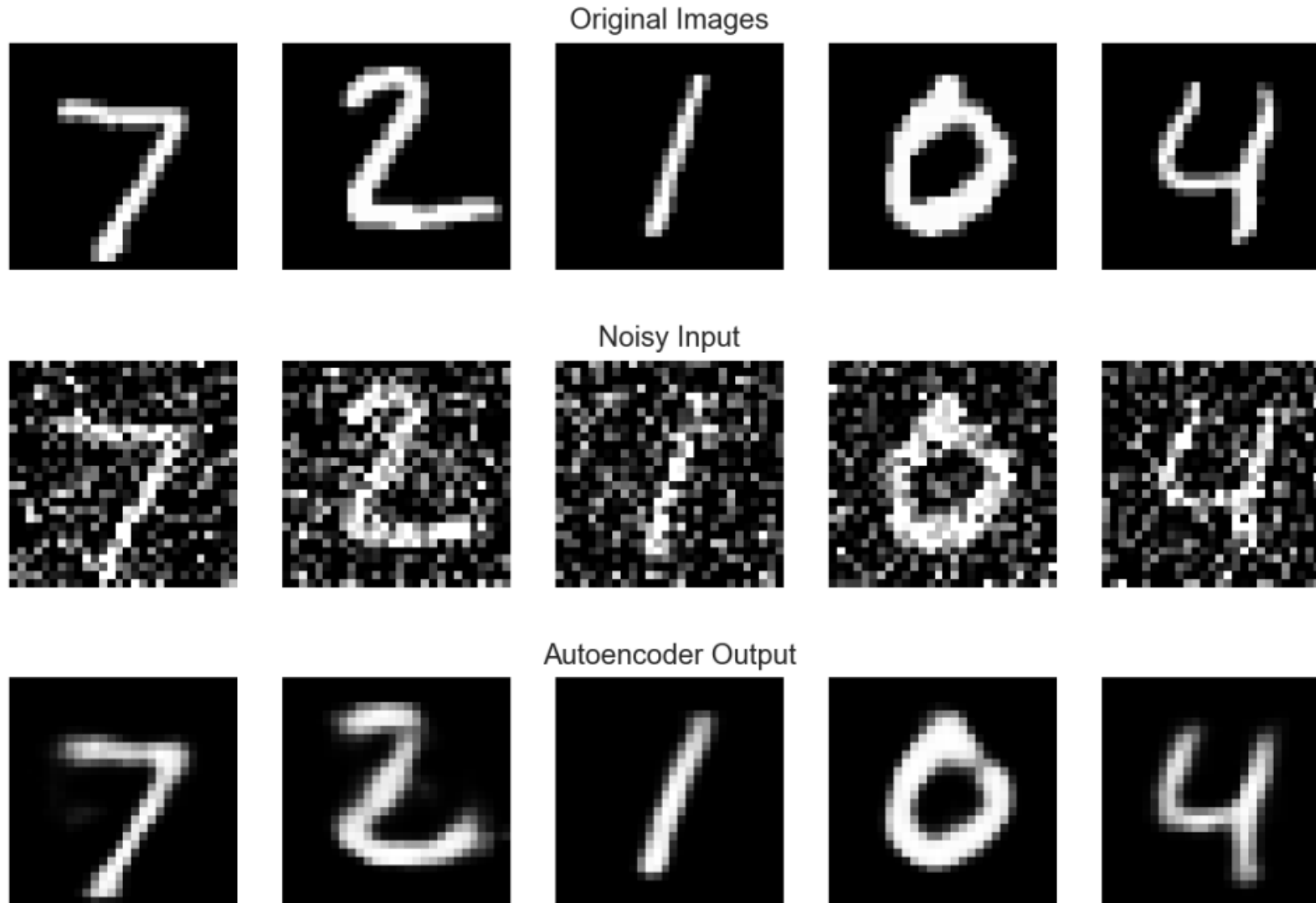
Similar Task

Denoising Autoencoders

- Add noise to the inputs, and train an autoencoder to recover this original
- Example Noise models:
 - ✓ **Gaussian Noise** ($\tilde{x} = x + z, z \sim N(0, \sigma^2 I)$)
 - ✓ **Masking noise** (randomly hiding parts of the input)
 - ✓ **Salt-and-pepper noise** (randomly replacing some input values with extreme values (minimum or maximum, like 0 and 1))



Sparse Autoencoders



Sparse Autoencoders

- An overcomplete architecture
- Regularize outputs of the hidden layer to enforce **sparsity**:

$$\hat{L}(x) = L(x, x') + \omega \Omega(z)$$

Loss function

Penalty for non-sparsity of $\mathbf{z}$

- $\Omega(\mathbf{z}) = \sum_i |z_i|$ and ReLU activation to force many zero output
- Can also measure the average activation of z_i across mini-batch and compare it to user-specified target sparsity value p via square error or **Kullback-Leibler (KL) divergence**:

$$p \log \frac{p}{q} + (1 - p) \log \frac{1-p}{1-q}$$

Average activation of z_i
over mini-batch

Contractive Autoencoders

$$\hat{L}(x) = L(x, x') + \omega \Omega(z)$$

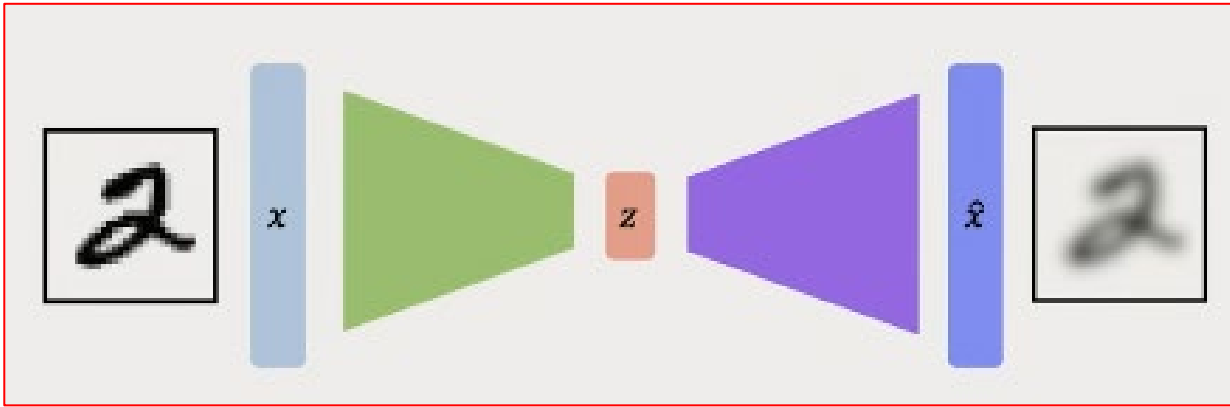
Loss function

Penalty

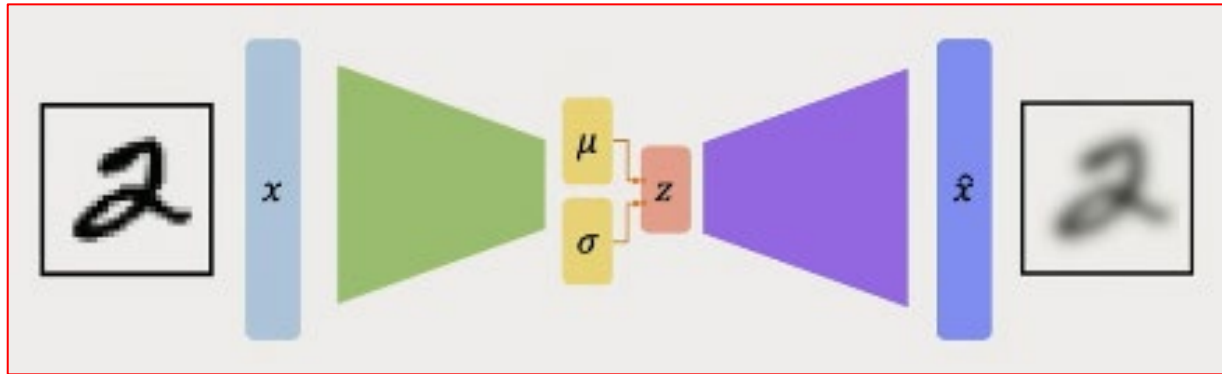
- Similar to a sparse autoencoder, but instead applies a different regularization term
- $\Omega(z) = \sum_{j=1}^m \sum_{i=1}^n \left(\frac{\partial z_i}{\partial x_j} \right)^2$, penalty for the large partial derivatives of encoder outputs w.r.t input values
- Makes the representation robust to small changes or noise in the input

Variational Autoencoder (VAEs)

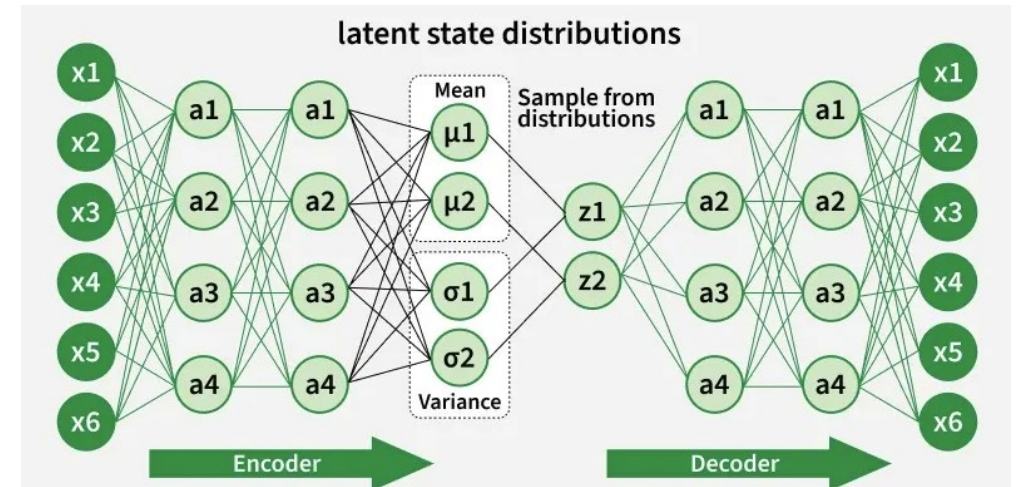
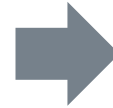
- VAE is an autoencoder that is also generative model
- VAE can generate new instances according to a probability distribution



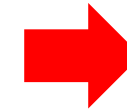
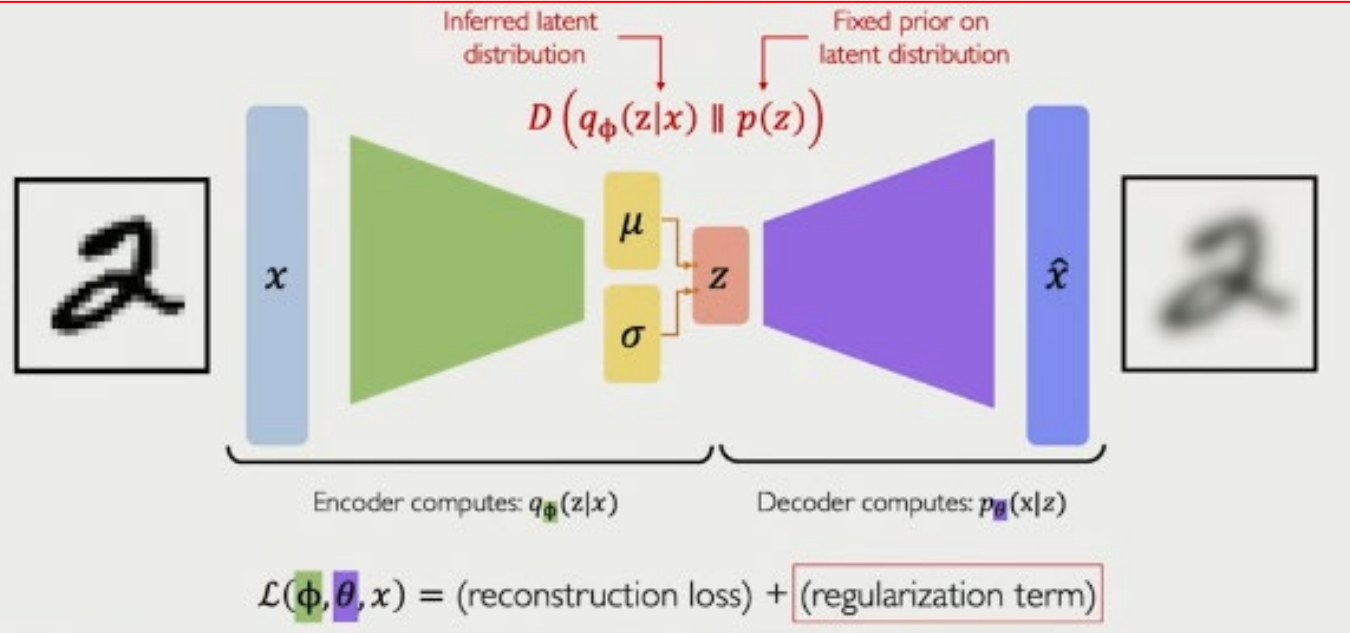
Autoencoder



Variational Autoencoder



Variational Autoencoder (VAEs)



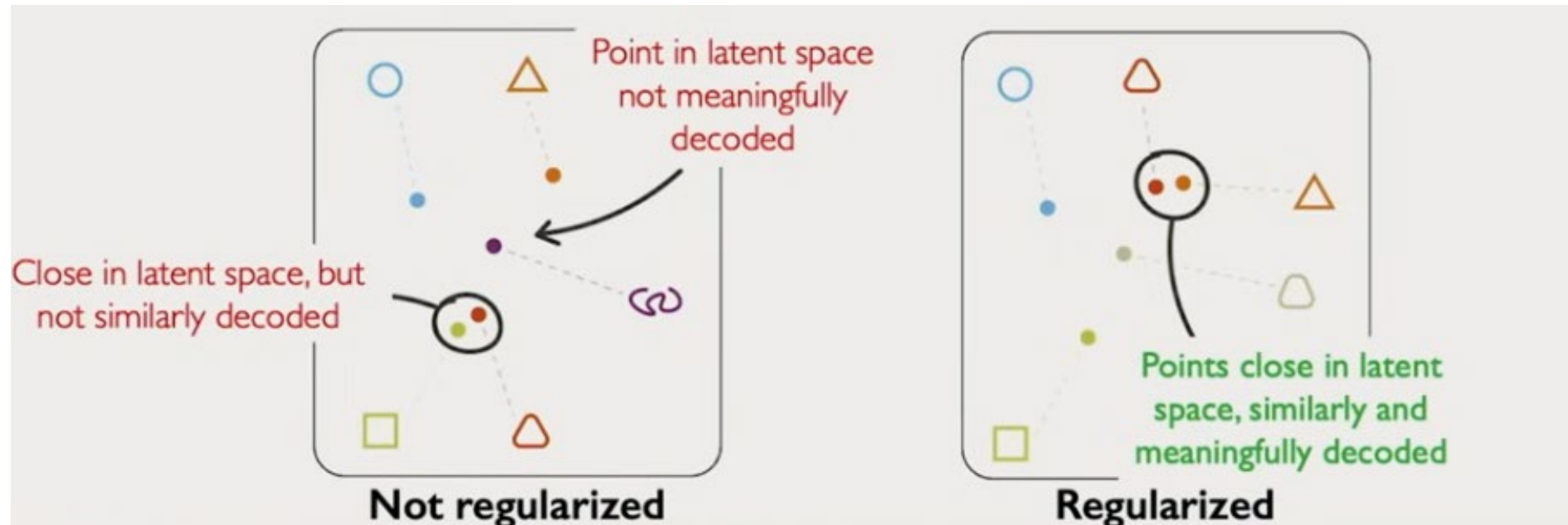
$$D(q_{\phi}(z|x) \parallel p(z)) = -\frac{1}{2} \sum_{j=0}^{k-1} (\sigma_j + \mu_j^2 - 1 - \log \sigma_j)$$

KL-divergence between the two distributions

- $p(z) = N(\mu = 0, \sigma^2 = 1)$
- Encourages encodings to distribute encodings evenly around the center of the latent space and penalize the network when it tries to cheat by clustering points in specific regions (i.e., by memorizing the data)

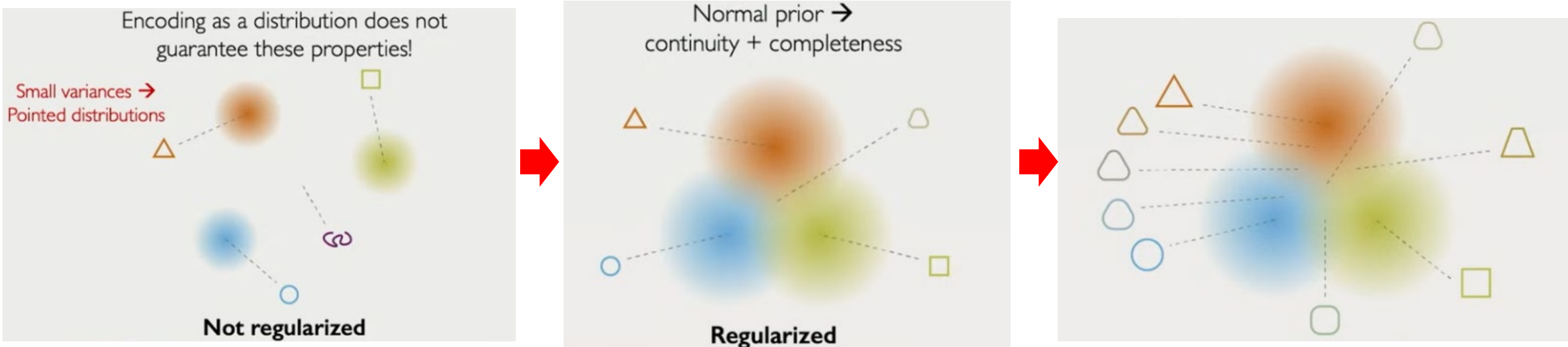
Variational Autoencoder (VAEs)

- **Continuity:** points that are close in the latent space $\rightarrow$ similar content after decoding
- **Completeness:** sampling from latent space $\rightarrow$ “meaningful” content after decoding



Variational Autoencoder (VAEs)

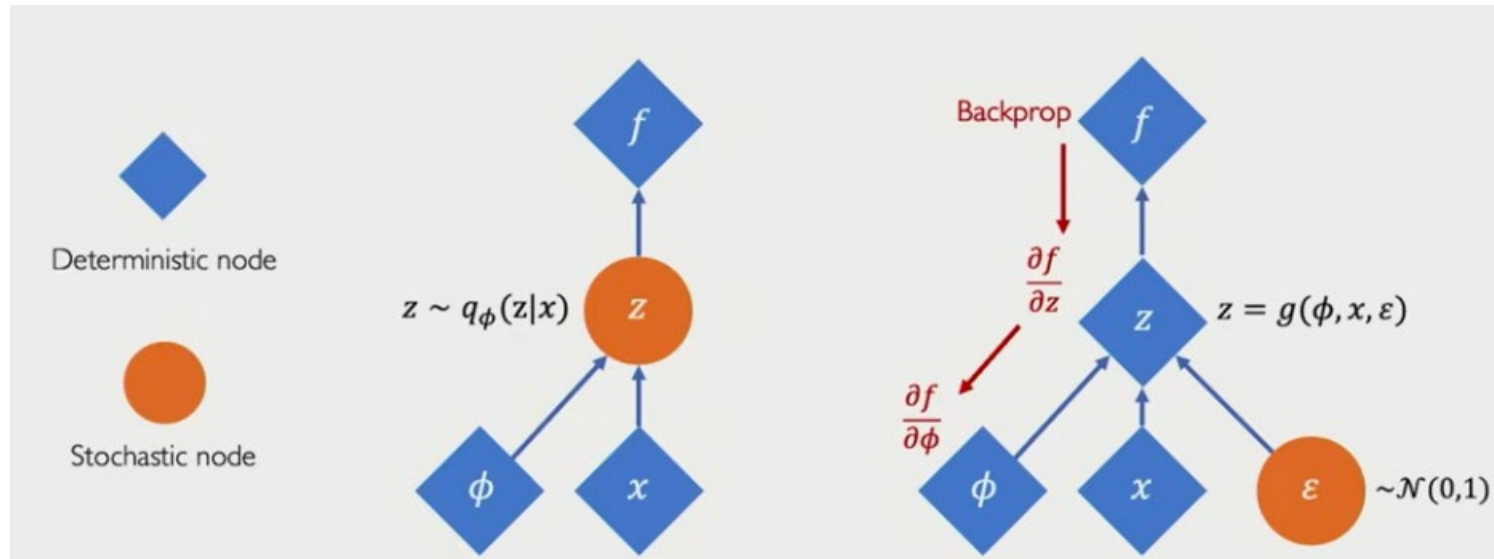
- **Continuity:** points that are close in the latent space $\rightarrow$ similar content after decoding
- **Completeness:** sampling from latent space $\rightarrow$ “meaningful” content after decoding



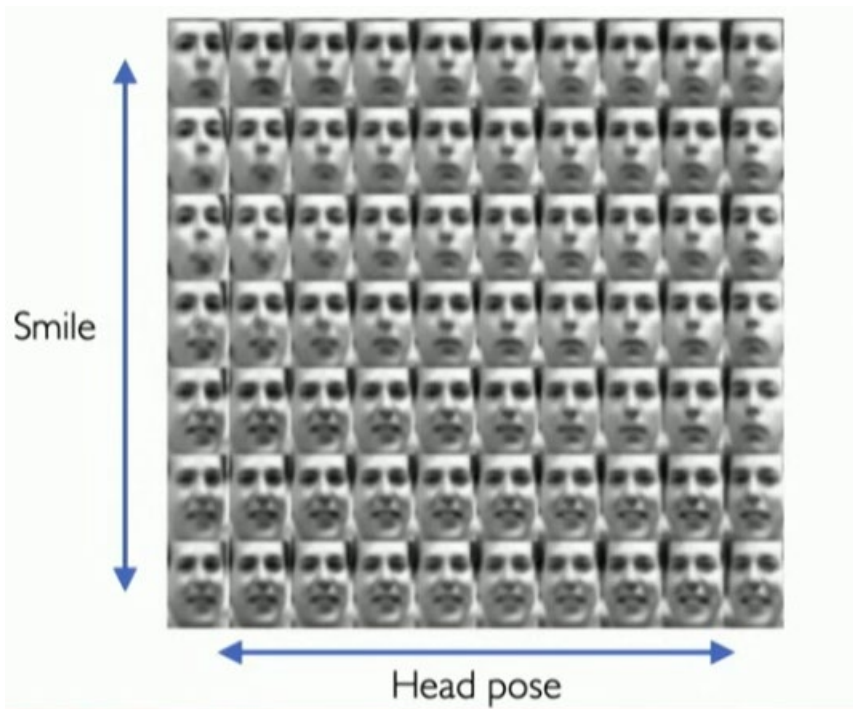
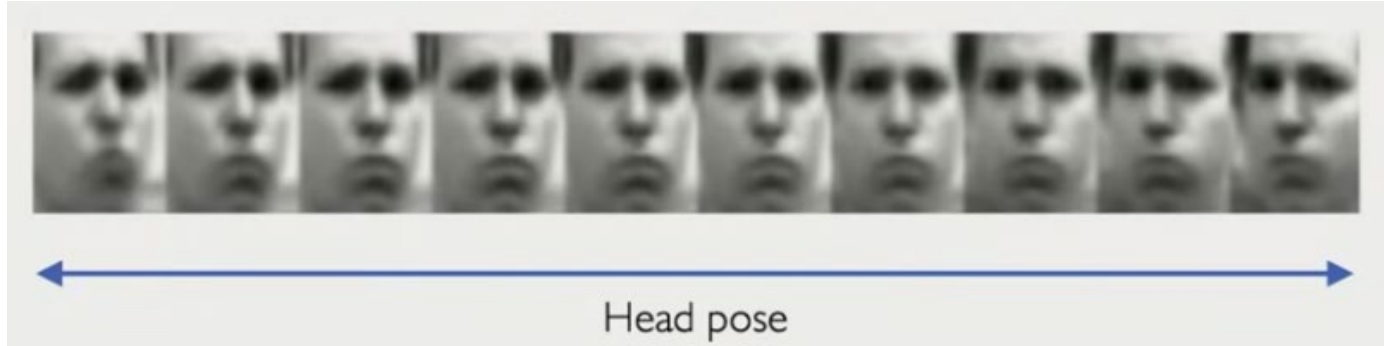
Variational Autoencoder (VAEs)

- As we have probabilistic latent space, we can not back propagate. Back propagation requires deterministic neural network layers
- The we can **reparametrize** the latent space
- Instead of $z \sim N(\mu, \sigma^2)$, we can consider the latent vector, z as a sum of: a fixed μ vector, and a fixed σ , scaled by random constants drawn from the prior distribution

$$z = \mu + \sigma \varepsilon, \quad \text{where } \varepsilon \sim N(0,1)$$



Variational Autoencoder (VAEs)



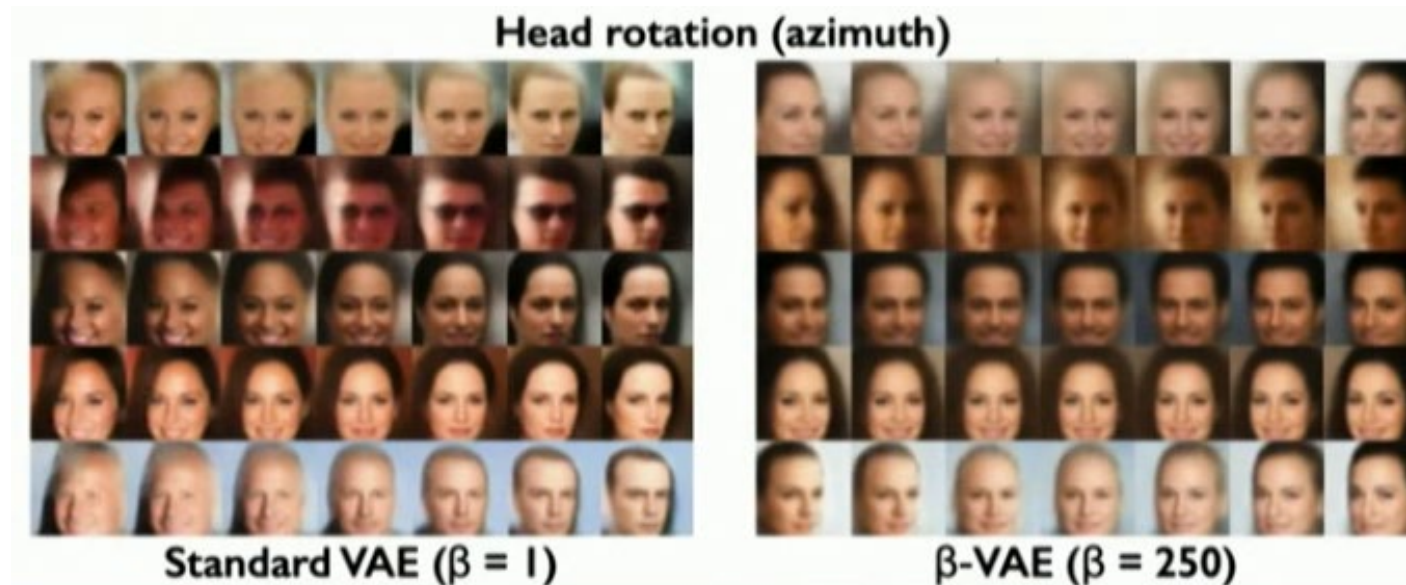
➤ **Note:** we want latent variables/features that are uncorrelated with each other

β - VAEs

➤ β - VAE loss:

$$\mathcal{L}(\theta, \phi; \mathbf{x}, \mathbf{z}, \beta) = \underbrace{\mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p_{\theta}(\mathbf{x}|\mathbf{z})]}_{\text{Reconstruction term}} - \underbrace{\beta D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}))}_{\text{Regularization term}}$$

Smile also
changing



Smile
relatively
constant

